SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and

Big Data Analytics **COURSE CODE: CSA15**

COURSE OUTCOME COVERED

CO5: Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)

Assessment Tool Weightage: 20%

Dave's Taxonomy: Precision (Level 3)

Question Count: 3 **Total Marks: 30**

Code of Conduct

I **A. Reddy Prasad (192472045)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student**: A. Reddy Prasad**

1. Uneven Input Splits and Idle Reducers

Likely design error:
The input data is highly unbalanced, or the input split/block size is poorly configured. This causes some mappers to receive much more data than others, leading to a **data-skew problem**. As a result, some reducers finish quickly while others remain overloaded or idle.

Corrective actions:

- Use appropriate HDFS block/input split sizes.
- Identify and remove data skew.
- Use a suitable partitioner to distribute keys evenly.
- Increase the number of reducers when required.
- Use combiners to reduce intermediate data.
- Balance the input files across HDFS.

---

2. HDFS Data Unavailability After One Node Failure

Probable design error:
The HDFS replication factor may be too low, such as **replication factor = 1**, or replicas may have been placed poorly. If the only copy of a block is on the failed DataNode, that data becomes unavailable.

Another possible issue is poor **NameNode design**, such as having only one NameNode without a properly configured standby NameNode.

Corrective actions:

- Set an appropriate replication factor, commonly **3** for important data.
- Ensure replicas are distributed across different DataNodes/racks.
- Use HDFS rack awareness.
- Configure **NameNode High Availability (HA)** with an active and standby NameNode.
- Regularly monitor missing or under-replicated blocks.

---

3. Duplicate Records in ETL/NiFi Workflow

Likely causes:

- The NiFi flow may process the same file or message more than once.
- The source system may resend records.

- Incorrect processor relationships or retry configurations may cause duplicate processing.
- No unique identifier or deduplication mechanism is used.
- The analytics database may not enforce uniqueness.

Recommended fixes:

- Use a unique record ID for every record.
- Configure NiFi processors and relationships correctly.
- Use appropriate retry and failure handling.
- Maintain checkpoints/state to avoid reprocessing.
- Implement deduplication before loading into the analytics store.
- Add **unique constraints/upsert or merge logic** in the target database.
- Design the ETL process to be **idempotent**, so processing the same data again does not create duplicates.

---

4. YARN Resource Starvation

Likely scheduling error:
The cluster may be using an inappropriate scheduler configuration or allocating too many resources to individual jobs. Without proper queue management, one user's jobs can consume most of the available CPU and memory, leaving insufficient resources for other users.

Better resource management approach:

- Configure **Capacity Scheduler** or **Fair Scheduler**.
- Create separate queues for different users or departments.
- Set minimum and maximum resource limits for queues.
- Configure CPU and memory resource allocation.
- Set application-level resource limits.
- Use queue priorities appropriately.
- Monitor cluster utilization through YARN ResourceManager.

Recommended approach:
Use **Capacity Scheduler with resource quotas and separate queues** when multiple users or departments need predictable resource allocation.

---

5. Backup Restores Outdated Data

Probable design flaw:
The backup system may be using **stale backups**, incorrect replication synchronization, or backups that are not versioned properly. If replication is asynchronous and the latest changes have not reached the backup before failure, recovery may restore an older version.

Corrective actions:

- Schedule backups frequently according to the required Recovery Point Objective (RPO).
- Use incremental backups along with periodic full backups.
- Maintain multiple backup versions/snapshots.
- Verify that replication is synchronized before relying on it for recovery.
- Regularly test backup restoration.
- Maintain backup metadata and timestamps.
- Follow a **3-2-1 backup strategy**: 3 copies of data, on 2 different storage types, with 1 copy stored off-site.

Result:
A properly designed replication and backup system should provide **current, recoverable, and verified data** even after failures.